

CSA1417- COMPILER DESIGN

Name: K. Ragavi

Register no: 192524364

LAB EXPERIMENT

Exp. No. 1

Develop a lexical Analyzer to identify identifiers, constants, operators using C program.

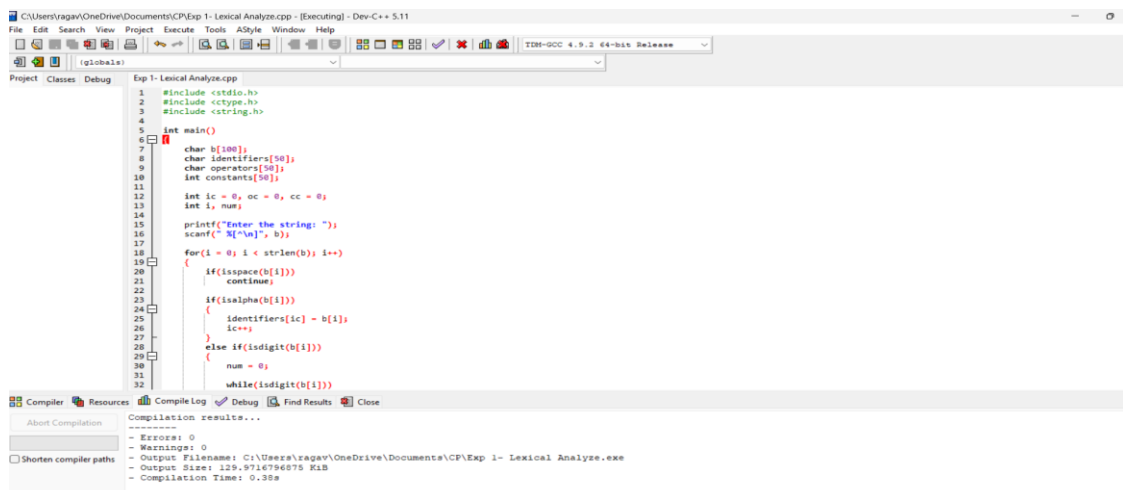
Aim

To develop a lexical analyzer to identify **identifiers, constants, and operators** using a C program.

Algorithm

1. Start the program.
2. Read the input string.
3. Scan the string character by character.
4. Identify alphabetic characters as **identifiers**.
5. Identify numeric values as **constants**.
6. Identify +, -, *, and = as **operators**.
7. Display the identified identifiers, constants, and operators.
8. Stop the program.

Input

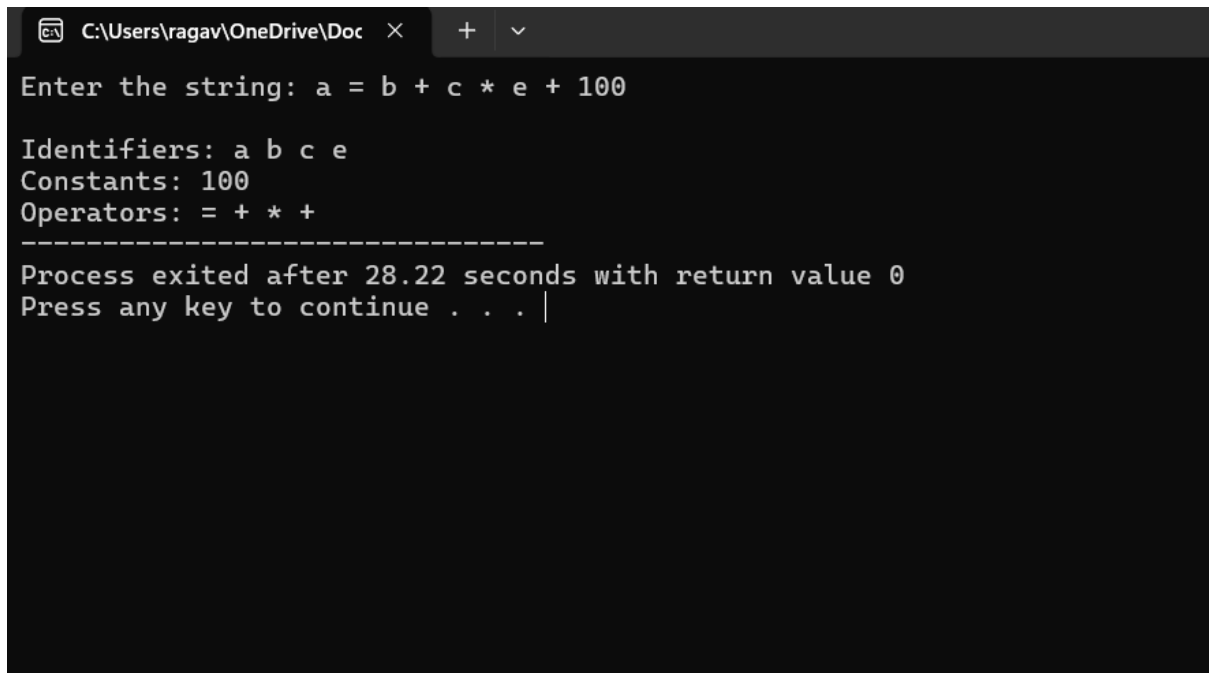


```
1 #include <stdio.h>
2 #include <ctype.h>
3 #include <string.h>
4
5 int main()
6 {
7     char b[100];
8     char identifiers[50];
9     char operators[50];
10    int constants[50];
11
12    int ic = 0, oc = 0, cc = 0;
13    int i, num;
14
15    printf("Enter the string: ");
16    scanf("%s", b);
17
18    for(i = 0; i < strlen(b); i++)
19    {
20        if(isspace(b[i]))
21            continue;
22
23        if(isalpha(b[i]))
24        {
25            identifiers[i] = b[i];
26            ic++;
27        }
28        else if(isdigit(b[i]))
29        {
30            num = 0;
31            while(isdigit(b[i]))
32            {
33                num = num * 10 + b[i] - '0';
34                i++;
35            }
36            constants[i] = num;
37            cc++;
38        }
39        else if(b[i] == '+' || b[i] == '-' || b[i] == '*' || b[i] == '=')
40        {
41            operators[i] = b[i];
42            oc++;
43        }
44    }
45
46    printf("Identifiers: ");
47    for(i = 0; i < ic; i++)
48        printf("%c", identifiers[i]);
49
50    printf("\nConstants: ");
51    for(i = 0; i < cc; i++)
52        printf("%d", constants[i]);
53
54    printf("\nOperators: ");
55    for(i = 0; i < oc; i++)
56        printf("%c", operators[i]);
57
58    return 0;
59 }
```

Compilation results...

```
- Errors: 0
- Warnings: 0
- Output Filename: C:\Users\ragavi\OneDrive\Documents\CP\Exp 1- Lexical Analyze.exe
- Output Size: 129,9716796875 KiB
- Compilation Time: 0.38s
```

Output



```
C:\Users\ragav\OneDrive\Doc  ×  +  ∨  
Enter the string: a = b + c * e + 100  
Identifiers: a b c e  
Constants: 100  
Operators: = + * +  
-----  
Process exited after 28.22 seconds with return value 0  
Press any key to continue . . . |
```

Result

Thus, the lexical analyzer was successfully developed in C to identify **identifiers**, **constants**, and **operators** from the given input string.

Exp. No. 2

Develop a lexical Analyzer to identify whether a given line is a comment or not using C

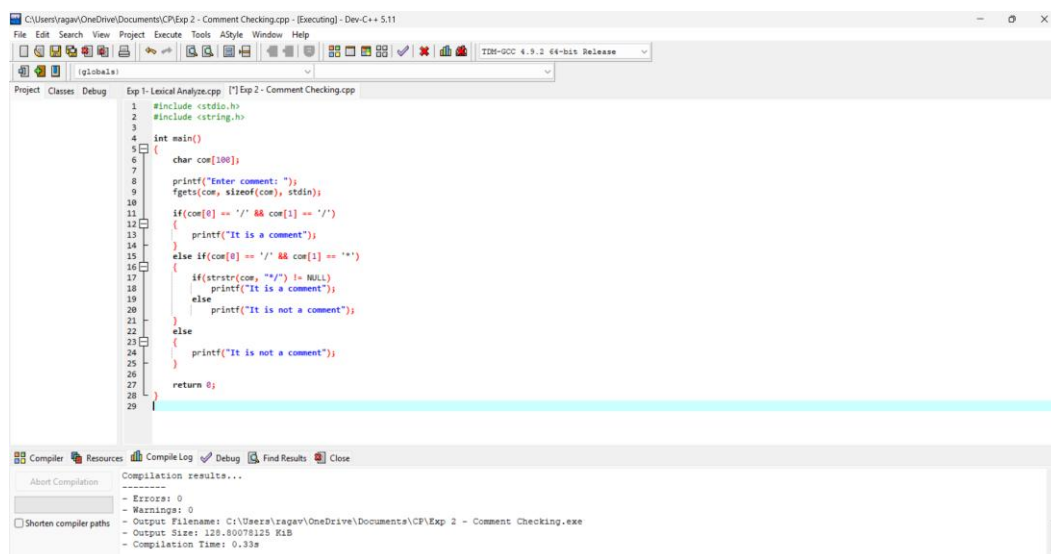
Aim

To develop a lexical analyzer to identify whether a given line is a **comment or not** using a C program.

Algorithm

1. Start the program.
2. Read the given line.
3. Check whether the line starts with //.
4. If it starts with //, identify it as a comment.
5. Otherwise, check whether it starts with /* and ends with */.
6. If the condition is satisfied, identify it as a comment.
7. Otherwise, display **It is not a comment**.
8. Stop the program.

Input

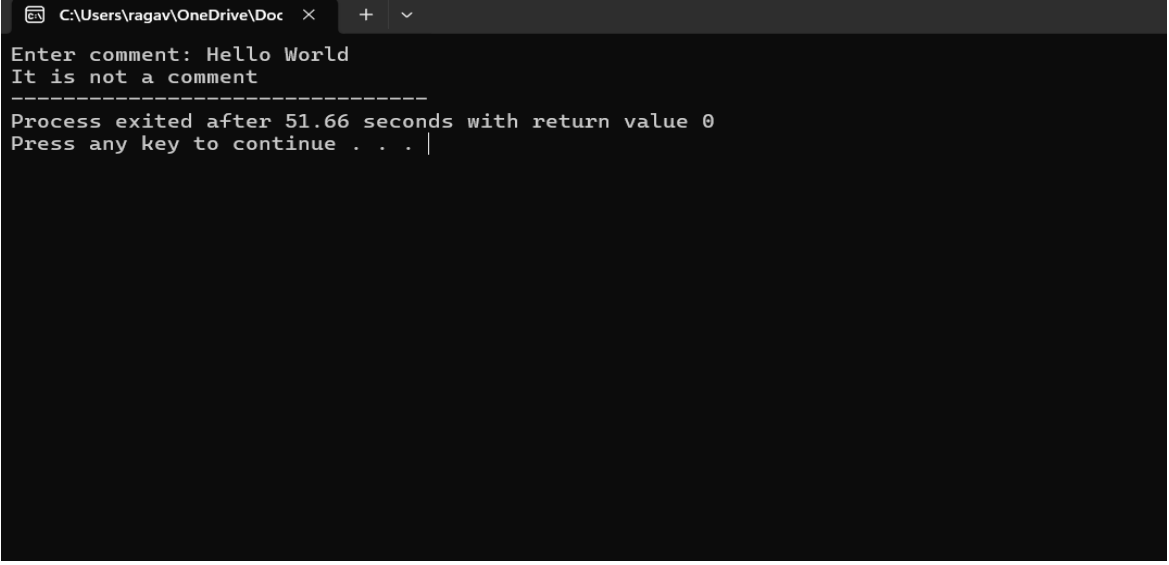


```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main()
5 {
6     char cce[100];
7     printf("Enter comment: ");
8     fgets(cce, sizeof(cce), stdin);
9
10    if(cce[0] == '/' && cce[1] == '/')
11    {
12        printf("It is a comment");
13    }
14    else if(cce[0] == '/' && cce[1] == '*')
15    {
16        if(strstr(cce, "*/") != NULL)
17            printf("It is a comment");
18        else
19            printf("It is not a comment");
20    }
21    else
22    {
23        printf("It is not a comment");
24    }
25
26    return 0;
27 }
28
29
```

Compilation results...

Errors: 0
Warnings: 0
Output Filename: C:\Users\ragar\OneDrive\Documents\CP\Exp 2 - Comment Checking.exe
Output Size: 129,800/70225 KiB
Compilation Time: 0.33s

Output



```
C:\Users\ragav\OneDrive\Doc >
Enter comment: Hello World
It is not a comment
-----
Process exited after 51.66 seconds with return value 0
Press any key to continue . . . |
```

Result

Thus, the lexical analyzer was successfully developed in C to identify whether the given line is a **comment** or **not**.